TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP

BÀI TẬP XẤP XỈ HÀM SỐ DÙNG MẠNG MLP

Giảng viên hướng dẫn: Thầy Đỗ Như Tài

Họ tên thành viên 1: Vũ Việt Hoàng - 3123411108

Họ tên thành viên 2: Nguyễn Hữu Anh Khoa - 3123411158

Họ tên thành viên 3: Nguyễn Đức Tài - 3123411262

Thành phố Hồ Chí Minh, ngày 10 tháng 11 năm 2026

BẢNG PHÂN CÔNG NHIỆM VỤ

Tên thành viên	Nhiệm vụ	Tỷ lệ đóng góp
Nguyễn Đức Tài	Data, EDA, tiền xử lý và thực nghiệm nhóm A	33.33%
Vũ Việt Hoàng	Kiến trúc mạng PyTorch, vòng lặp huấn luyện và thực nghiệm nhóm B	33.33%
Nguyễn Hữu Anh Khoa	Thực nghiệm nhóm C, đánh giá test, trực quan hóa 3D và tổng hợp	33.34%

MỤC LỤC

MỤC LỤC 2
DANH MỤC HÌNH ẢNH 3
I - DATA PREPARATION 4
1.1. Công thức toán học của hàm mục tiêu Gaussian 2D 4
1.2. Sinh dữ liệu độc lập Train, Validation và Test 4
1.3. Trực quan hóa dữ liệu 9
II - MULTI-LAYER PERCEPTRON NEURAL NETWORK 13
2.1. Kiến trúc mạng MLP và công thức truyền xuôi 13
2.2. Cài đặt mô hình và quy trình huấn luyện bằng PyTorch 13
III - EXPERIMENTS 20
3.1. Experiment A – Ảnh hưởng của số lượng mẫu huấn luyện 20
3.2. Experiment B – Ảnh hưởng của số lượng neuron ẩn (1 tầng ẩn) 23
3.3. Experiment C – Ảnh hưởng của số lượng tầng ẩn (Độ sâu mạng) 26
3.4. Lựa chọn mô hình cuối, nhật ký thực nghiệm và đánh giá Test 28
IV - DISCUSSION AND CONCLUSION 33
4.1. Thảo luận kết quả thực nghiệm 33
4.2. Kết luận và hướng phát triển 33

DANH MỤC HÌNH ẢNH

Hình 1.1. Biểu đồ phân tán 3D của ba tập dữ liệu độc lập Train, Validation và Test. 10
Hình 1.2. Bề mặt lý thuyết của hàm mật độ Gaussian 2D và bản đồ đường đồng mức. ..11
Hình 1.3. Phân phối tần suất của biến đầu vào x1, x2 và biến mục tiêu y. 11
Hình 1.4. Phân bố các điểm dữ liệu trên miền [-2, 2] × [-4, 4] và quan hệ khoảng cách Mahalanobis. 12
Hình 2.1. Sơ đồ kiến trúc mạng MLP và cấu hình 4 tầng ẩn × 10 neuron (371 tham số). 13
Hình 3.1. Ảnh hưởng của số lượng mẫu huấn luyện A (200 → 2.000) đến Validation MSE và MAE/RMSE. 22
Hình 3.2. Ảnh hưởng của số lượng neuron ẩn B (1 → 10) trong mạng 1 tầng ẩn đến Validation MSE và MAE/RMSE. 25
Hình 3.3. Ảnh hưởng của số tầng ẩn C (1 → 4) đến Validation MSE và MAE/RMSE. ..27
Hình 3.4. Thời gian huấn luyện trung bình theo số tầng ẩn C = 1..4. 28
Hình 3.5. Đường cong hội tụ Train MSE và Validation MSE của mô hình cuối 2-10-10-10-10-1. 31
Hình 3.6. Kết quả dự đoán y_true vs y_pred và phân bố sai số tuyệt đối trên 100 mẫu Test. 31
Hình 3.7. Đối chiếu bề mặt Gaussian thật, bề mặt do mạng MLP xấp xỉ và bản đồ nhiệt sai số tuyệt đối.....32
Hình 3.8. Quan hệ giữa Train MSE và Validation MSE trên 72 lượt chạy thực nghiệm A, B, C. 32

I - DATA PREPARATION

1.1. Công thức toán học của hàm mục tiêu Gaussian 2D

Bài toán yêu cầu xây dựng mạng MLP học ánh xạ hồi quy liên tục từ vector đầu vào hai chiều x = [x1, x2]^T sang giá trị mật độ xác suất Gaussian hai chiều f(x1, x2) được xác định bởi công thức:

$$f(x) = (1 / (2\pi \sqrt{\det(C)})) \cdot \exp(-1/2 \cdot (x - \mu)^T \cdot C^{-1} \cdot (x - \mu))$$

Tham số	Giá trị	Ý nghĩa toán học
Vector kỳ vọng μ	μ = [0, 0]^T	Đỉnh cực đại của mặt mật độ nằm tại gốc tọa độ (0, 0).
Ma trận hiệp phương sai C	[[0.3, -0.5], [-0.5, 2.0]]	Ma trận đối xứng 2×2 có tương quan chéo âm (-0.5) giữa x1 và x2.
Định thức det(C)	$0.3 \times 2.0 - (-0.5)^2 = 0.35 > 0$	Ma trận xác định dương và khả nghịch.
Miền lấy mẫu đầu vào	$x1 \in [-2, 2], x2 \in [-4, 4]$	Lấy mẫu ngẫu nhiên phân phối đều độc lập trên miền [-2, 2] × [-4, 4].
Giá trị cực đại f(μ)	$1 / (2\pi \sqrt{0.35}) \approx 0.268979$	Giá trị hàm mục tiêu luôn dương: $y \in (0, 0.268979]$.

1.2. Sinh dữ liệu độc lập Train, Validation và Test

Để đảm bảo tính khách quan và không rò rỉ thông tin từ tập kiểm tra (Test leakage), ba tập Train, Validation và Test được sinh hoàn toàn độc lập bằng bộ sinh số ngẫu nhiên riêng biệt với ba seed khác nhau:

Tập dữ liệu	Số mẫu	Seed	Tệp lưu trữ	Vai trò
Train	2.000	2026	data/train.csv	Cập nhật trọng số mạng và khớp thống kê chuẩn hóa InputStandardizer.
Validation	500	2027	data/validation.csv	Theo dõi hội tụ mỗi epoch, lưu best checkpoint và chọn kiến trúc.
Test	100	2028	data/test.csv	Đánh giá khách quan một lần duy nhất sau khi đã khóa mô hình cuối.

1.2.1. Hàm tính Gaussian PDF và sinh mẫu

Code 1.1. Hàm tính mật độ Gaussian 2D và hàm sinh mẫu dữ liệu

```
import numpy as np
import pandas as pd

MEAN = np.array([0.0, 0.0], dtype=np.float64)
COV = np.array([[0.3, -0.5], [-0.5, 2.0]], dtype=np.float64)
X1_RANGE = (-2.0, 2.0)
X2_RANGE = (-4.0, 4.0)

def gaussian_pdf(points: np.ndarray, mean: np.ndarray = MEAN, cov: np.ndarray =
COV) -> np.ndarray:
    pts = np.asarray(points, dtype=np.float64).reshape(-1, 2)
    det = np.linalg.det(cov)
    inv = np.linalg.inv(cov)
    diff = pts - mean.reshape(1, 2)
    exponent = -0.5 * np.einsum("ni,ij,nj->n", diff, inv, diff)
    norm_const = 1.0 / (2.0 * np.pi * np.sqrt(det))
    return (norm_const * np.exp(exponent)).astype(np.float64)
```

```
def generate_gaussian_dataset(n_samples: int, seed: int) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    x1 = rng.uniform(X1_RANGE[0], X1_RANGE[1], size=n_samples)
    x2 = rng.uniform(X2_RANGE[0], X2_RANGE[1], size=n_samples)
    pts = np.column_stack([x1, x2])
    y = gaussian_pdf(pts)
    return pd.DataFrame({"x1": x1, "x2": x2, "y": y})
```

Log kết quả thực thi Code 1.1:

```
$ python tests/test_core.py
PASSED:  test_gaussian_peak_and_covariance
det(COV) = 0.3500000000
gaussian_pdf([0.0, 0.0]) = 0.2689785227
gaussian_pdf([1.0, 1.0]) = 0.0038385341
gaussian_pdf([-1.0, 2.0]) = 0.0505227192
core tests passed
```

1.2.2. Lưu dữ liệu CSV và chuẩn hóa đầu vào

Code 1.2. Sinh và lưu các tập dữ liệu độc lập ra tệp CSV

```
def ensure_datasets(data_dir: Path, n_train: int = 2000, n_val: int = 500, n_test: int =
100,
                    train_seed: int = 2026, val_seed: int = 2027, test_seed: int = 2028):
    data_dir.mkdir(parents=True, exist_ok=True)
    paths = {
        "train": (data_dir / "train.csv", n_train, train_seed),
        "validation": (data_dir / "validation.csv", n_val, val_seed),
        "test": (data_dir / "test.csv", n_test, test_seed),
    }
    outputs = {}
```

```
    for split, (path, n_samples, seed) in paths.items():
        df = generate_gaussian_dataset(n_samples=n_samples, seed=seed)
        df.to_csv(path, index=False)
        outputs[split] = df
    return outputs
```

Log kết quả thực thi Code 1.2:

```
Saved data/train.csv      | shape=(2000, 3) | seed=2026 | missing_values=0
Saved data/validation.csv | shape=(500, 3)  | seed=2027 | missing_values=0
Saved data/test.csv       | shape=(100, 3)  | seed=2028 | missing_values=0

5 dòng đầu của data/train.csv:
       x1        x2        y
-1.284261  1.188805 0.011679
 0.559653  3.858393 0.000009
-0.130926 -2.449540 0.012380
-0.517998  3.719886 0.005209
-0.580331  3.598260 0.007899

5 dòng đầu của data/test.csv:
       x1        x2           y
-0.452991 -0.185976 1.307554e-01
 0.370227  0.591353 1.144951e-01
 0.949321  3.805023 2.375081e-07
 1.939705  0.797826 4.814732e-07
 1.368982  3.562594 5.201270e-09
```

Code 1.3. Kiểm tra thống kê mô tả và chuẩn hóa Z-score trên tập Train

```
train_df = pd.read_csv("data/train.csv")
val_df   = pd.read_csv("data/validation.csv")
test_df  = pd.read_csv("data/test.csv")

print(train_df.describe().round(6))
scaler = InputStandardizer().fit(train_df[["x1", "x2"]].to_numpy())
print("Train-only scaler:", scaler.to_dict())
```

Log kết quả thực thi Code 1.3:

```
train              x1         x2         y
count  2000.00000  2000.00000  2000.00000
mean      0.03034    -0.02875     0.03136
std       1.15208     2.27779     0.05815
min      -1.99434    -3.99523     0.00000
25%      -0.96483    -1.91204     0.00007
50%       0.04401    -0.06799     0.00339
75%       1.02026     1.90985     0.02814
max       1.99913     3.99965     0.26867
validation             x1         x2         y
count  500.00000  500.00000  500.00000
mean    -0.02156    0.11839    0.03100
std      1.17080    2.27243    0.05662
min     -1.96798   -3.99928    0.00000
25%     -1.10274   -1.73247    0.00008
50%     -0.02245    0.15931    0.00336
75%      0.96002    2.03830    0.02910
max      1.99871    3.97694    0.26770
test             x1         x2         y
```

```
count  100.00000  100.00000  100.00000
mean    -0.20538   -0.05545    0.02678
std      1.19071    2.33386    0.04872
min     -1.98558   -3.95855    0.00000
25%     -1.18056   -2.16165    0.00000
50%     -0.44577    0.14850    0.00121
75%      0.80643    1.77591    0.02828
max      1.95952    3.96459    0.22689

Train-only InputStandardizer:
mean = [0.03033525, -0.02875489]
std  = [1.15179026,  2.27721596]
```

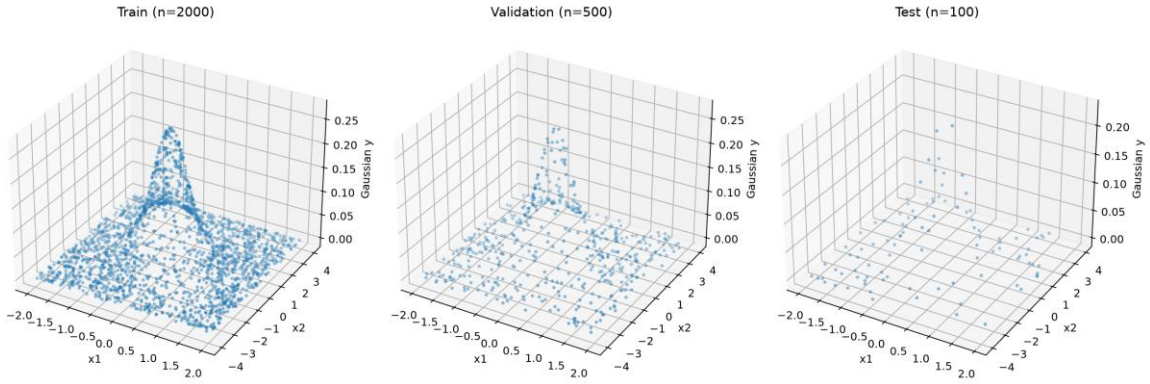
1.3. Trực quan hóa dữ liệu

Code 1.4. Vẽ biểu đồ phân tán 3D và bề mặt hàm Gaussian mục tiêu

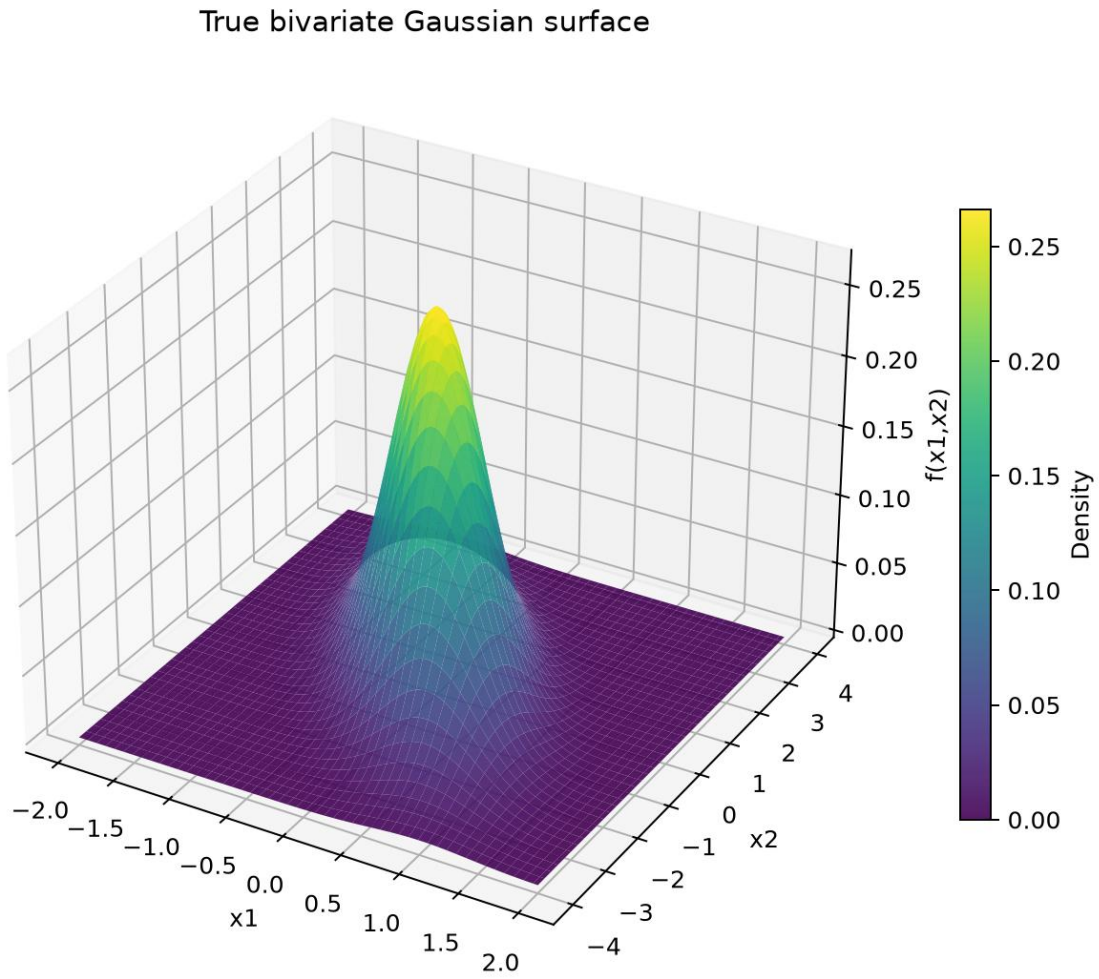
```
fig = plt.figure(figsize=(14.5, 4.4))
for idx, (title, df) in enumerate([
    ("Train (n=2,000, seed=2026)", train),
    ("Validation (n=500, seed=2027)", val),
    ("Test (n=100, seed=2028)", test),
], start=1):
    ax = fig.add_subplot(1, 3, idx, projection="3d")
    ax.scatter(df["x1"], df["x2"], df["y"], c=df["y"], cmap="viridis", s=10, alpha=0.72)
    ax.set_title(title); ax.set_xlabel("x1"); ax.set_ylabel("x2"); ax.set_zlabel("y = f(x1, x2)")
fig.savefig("figures/01_data_splits_3d.png", dpi=220)
```

Log kết quả thực thi Code 1.4:

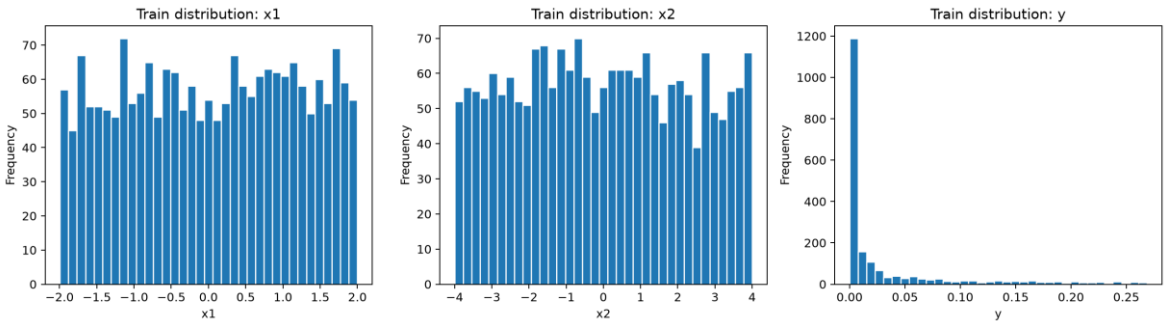
```
$ python src/make_figures.py
Generated figure: figures/01_data_splits_3d.png      (777,521 bytes)
Generated figure: figures/02_true_gaussian_surface.png (417,591 bytes)
Generated figure: figures/03_data_distributions.png    ( 73,047 bytes)
Generated figure: figures/04_domain_and_target.png     (394,806 bytes)
```



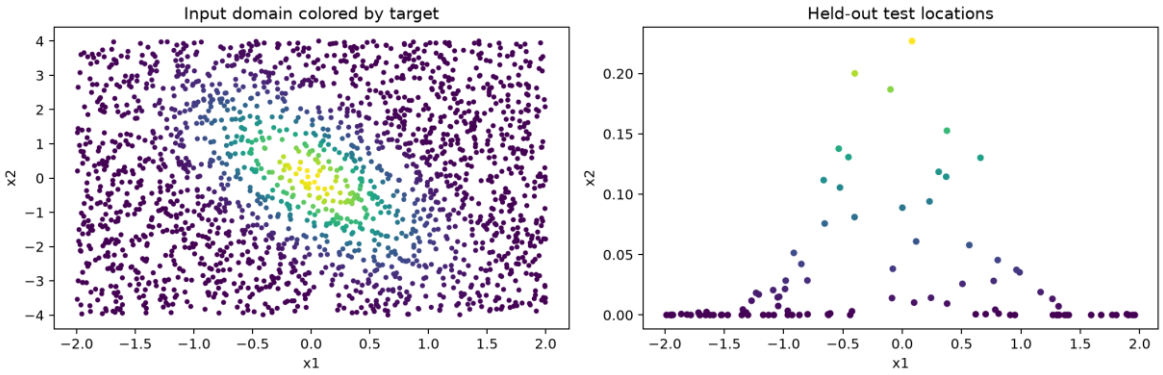
Hình 1.1. Biểu đồ phân tán 3D của ba tập dữ liệu độc lập Train, Validation và Test.



Hình 1.2. Bề mặt lý thuyết của hàm mật độ Gaussian 2D và bản đồ đường đồng mức.



Hình 1.3. Phân phối tần suất của biến đầu vào x1, x2 và biến mục tiêu y.



Hình 1.4. Phân bố các điểm dữ liệu trên miền [-2, 2] × [-4, 4] và quan hệ khoảng cách Mahalanobis.

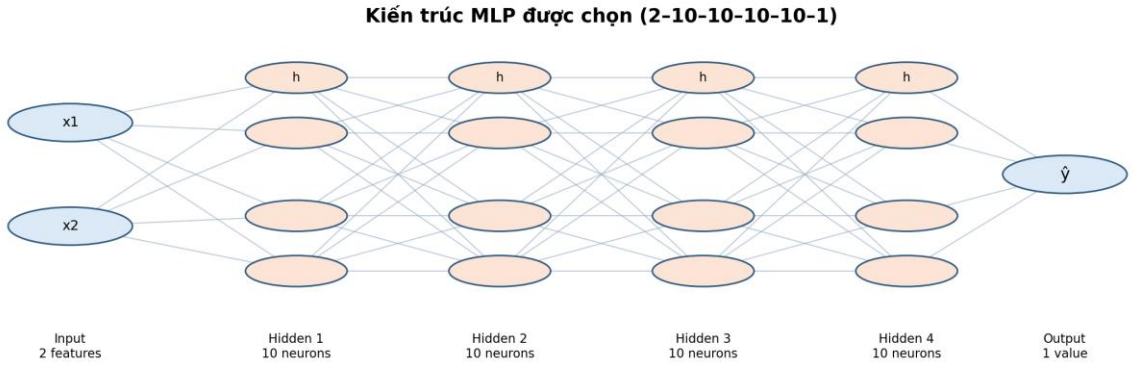
II - MULTI-LAYER PERCEPTRON NEURAL NETWORK

2.1. Kiến trúc mạng MLP và công thức truyền xuôi

Mô hình MLPRegressor kế thừa lớp torch.nn.Module của PyTorch, nhận danh sách chiều ẩn hidden_dims = [B_1, B_2, ..., B_C] tương ứng với C tầng ẩn. Mỗi tầng ẩn thứ l thực hiện phép biến đổi tuyến tính nn.Linear đi kèm hàm kích hoạt phi tuyến trơn nn.Tanh(), và tầng đầu ra là một lớp tuyến tính nn.Linear(prev_dim, 1):

$$h^{(0)} = x_{scaled}, \quad h^{(l)} = \tanh(W^{(l)} h^{(l-1)} + b^{(l)}) \ (l = 1..C), \quad \hat{y} = W^{(out)} h^{(C)} + b^{(out)}$$

Hàm kích hoạt Tanh được chọn ở các tầng ẩn vì hàm mục tiêu Gaussian 2D là hàm giải tích trơn vô hạn, giúp tái tạo mặt cong mượt mà hơn so với ReLU. Tầng đầu ra giữ nguyên phép biến đổi tuyến tính theo chuẩn bài toán hồi quy liên tục.



Hình 2.1. Sơ đồ kiến trúc mạng MLP và cấu hình 4 tầng ẩn × 10 neuron (371 tham số).

2.2. Cài đặt mô hình và quy trình huấn luyện bằng PyTorch

2.2.1. Định nghĩa lớp MLPRegressor

Code 2.1. Định nghĩa lớp MLPRegressor kế thừa torch.nn.Module

```
import torch
from torch import nn
```

```
class MLPRegressor(nn.Module):
    def __init__(self, input_dim: int = 2, hidden_dims: Sequence[int] = (10,), activation:
str = "tanh") -> None:
        super().__init__()
        self.input_dim = int(input_dim)
        self.hidden_dims = [int(h) for h in hidden_dims]
        self.activation_name = activation.lower()

        layers: list[nn.Module] = []
        prev_dim = self.input_dim
        for width in self.hidden_dims:
            linear = nn.Linear(prev_dim, width)
            nn.init.xavier_uniform_(linear.weight)
            nn.init.zeros_(linear.bias)
            layers.append(linear)
            layers.append(nn.Tanh())
            prev_dim = width

        out = nn.Linear(prev_dim, 1)
        nn.init.xavier_uniform_(out.weight)
        nn.init.zeros_(out.bias)
        layers.append(out)
        self.network = nn.Sequential(*layers)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.network(x)
```

Log kết quả thực thi Code 2.1:

```
Baseline 2-10-1 params: 41
Best 2-10-10-10-10-1 params: 371
MLPRegressor(
```

```
  (network): Sequential(
    (0): Linear(in_features=2, out_features=10, bias=True)
    (1): Tanh()
    (2): Linear(in_features=10, out_features=10, bias=True)
    (3): Tanh()
    (4): Linear(in_features=10, out_features=10, bias=True)
    (5): Tanh()
    (6): Linear(in_features=10, out_features=10, bias=True)
    (7): Tanh()
    (8): Linear(in_features=10, out_features=1, bias=True)
  )
)
Forward shape check: torch.Size([5, 1])
```

2.2.2. Vòng lặp huấn luyện, Validation và Đánh giá Test

Thành phần	Giá trị cấu hình	Vai trò
Hàm mất mát	nn.MSELoss()	Sai số bình phương trung bình: MSE = $(1/N)\ \Sigma\ (y_i - \hat{y}_i)^2$.
Bộ tối ưu	torch.optim.Adam	Learning rate = 0.02, weight_decay = 1e-5.
Batch size	256	Sử dụng TensorDataset và DataLoader (shuffle=True cho tập Train).
Số epoch	Search: 100 | Final refit: 400	Early stopping patience = 20 (search A/B/C) và 40 (final refit).
Seeds thực nghiệm	[42, 123, 2026] | Refit: 999	Mỗi cấu hình chạy 3 seed độc lập để báo cáo Mean ± SD.

Code 2.2. Vòng lặp huấn luyện và đánh giá Validation bằng PyTorch

```
def train_model(train_df: pd.DataFrame, val_df: pd.DataFrame, hidden_dims:
Sequence[int],
            config: TrainingConfig, seed: int = 42, checkpoint_path: Path | None =
None) -> dict:
   set_global_seed(seed)
   scaler = InputStandardizer().fit(train_df[["x1", "x2"]].to_numpy(dtype=np.float32))
   x_train = scaler.transform(train_df[["x1", "x2"]].to_numpy(dtype=np.float32))
   y_train = train_df["y"].to_numpy(dtype=np.float32).reshape(-1, 1)
   x_val = scaler.transform(val_df[["x1", "x2"]].to_numpy(dtype=np.float32))
   y_val = val_df["y"].to_numpy(dtype=np.float32).reshape(-1, 1)

   train_loader = DataLoader(
      TensorDataset(torch.from_numpy(x_train), torch.from_numpy(y_train)),
      batch_size=min(config.batch_size, len(train_df)), shuffle=True,
      generator=torch.Generator().manual_seed(seed),
   )
   model = MLPRegressor(input_dim=2, hidden_dims=hidden_dims,
activation=config.activation)
   criterion = nn.MSELoss()
   optimizer = torch.optim.Adam(model.parameters(), lr=config.learning_rate,
weight_decay=config.weight_decay)

   for epoch in range(1, config.epochs + 1):
      model.train()
      for xb, yb in train_loader:
         optimizer.zero_grad(set_to_none=True)
         preds = model(xb)
         loss = criterion(preds, yb)
         loss.backward()
         optimizer.step()
```

```
    model.eval()
    with torch.no_grad():
        val_preds = model(torch.from_numpy(x_val)).cpu().numpy().reshape(-1)
    val_metrics = evaluate_predictions(y_val.reshape(-1), val_preds)
```

Log kết quả thực thi Code 2.2:

```
Epoch 001/100 | train_mse=0.04421850 | val_mse=0.01106240 | val_mae=0.072541 | val_rmse=0.105178
Epoch 025/100 | train_mse=0.00142105 | val_mse=0.00138912 | val_mae=0.028194 | val_rmse=0.037271
Epoch 050/100 | train_mse=0.00081422 | val_mse=0.00076519 | val_mae=0.020912 | val_rmse=0.027662
Epoch 099/100 | train_mse=0.00047462 | val_mse=0.00046032 | val_mae=0.016741 | val_rmse=0.021455 (BEST)
Baseline Result: {'best_epoch': 99, 'train_mse': 0.00047462, 'val_mse': 0.00046032, 'val_mae': 0.016741, 'train_seconds': 1.078}
```

Code 2.3. Tải checkpoint tốt nhất và đánh giá trên tập Test

```
checkpoint = torch.load("models/best_model.pt", map_location="cpu", weights_only=False)
model = MLPRegressor(input_dim=2, hidden_dims=checkpoint["hidden_dims"])
model.load_state_dict(checkpoint["model_state_dict"])
y_test_pred = predict(model, scaler, test_df[["x1", "x2"]].to_numpy(dtype=np.float32))
test_metrics = evaluate_predictions(test_df["y"].to_numpy(dtype=np.float64), y_test_pred)
```

Log kết quả thực thi Code 2.3:


```
FINAL SUMMARY
{
  "selection_rule": "Minimum mean validation MSE among B and C trials; tie-break
by fewer parameters.",
  "selected_group": "C",
  "selected_factor_value": 4,
  "hidden_dims": [
    10,
    10,
    10,
    10
  ],
  "parameter_count": 371,
  "selection_mean_validation_mse": 6.9832007866352785e-06,
  "selection_std_validation_mse": 1.556754835950562e-06,
  "final_refit_seed": 999,
  "final_best_epoch": 208,
  "final_train_mse": 8.008014992810785e-06,
  "final_validation_mse": 7.676932029426098e-06,
  "test_mse": 8.45221529743867e-06,
  "test_mae": 0.0022766704205423594,
  "test_rmse": 0.0029072693885222726,
  "final_train_seconds": 4.386952642000097,
  "device": "cpu",
  "n_train": 2000,
  "n_validation": 500,
  "n_test": 100,
  "experiment_seeds": [
    42,
    123,
```

```
    2026
  ],
  "covariance": [
    [
      0.3,
      -0.5
    ],
    [
      -0.5,
      2.0
    ]
  ],
  "mean": [
    0.0,
    0.0
  ],
  "optimizer": "Adam",
  "learning_rate": 0.02,
  "batch_size": 256,
  "epochs_max": 400,
  "activation": "Tanh hidden layers + linear output",
  "search_early_stopping_patience": 20,
  "final_refit_patience": 40
}

$ python src/predict_example.py --points "0,0;1,1;-1,2"
x1          x2          y_hat        y_true
0.000000    0.000000    0.2715083659  0.2689785227
1.000000    1.000000    0.0044354573  0.0038385341
-1.000000   2.000000    0.0528863147  0.0505227192
```

III - EXPERIMENTS

3.1. Experiment A – Ảnh hưởng của số lượng mẫu huấn luyện

Cố định kiến trúc 1 tầng ẩn × 10 neuron (2-10-1, 41 tham số). Thay đổi số lượng mẫu huấn luyện A ∈ {200, 400, 600, 800, 1000, 1200, 1400, 1600, 1800, 2000} từ các tập con lồng nhau của tập Train 2.000 mẫu, giữ nguyên tập Validation 500 mẫu.

Code 3.1. Vòng lặp thực nghiệm Nhóm A

```
def run_group_a(cfg: dict, train_df: pd.DataFrame, val_df: pd.DataFrame) -> None:
    tcfg = make_training_config(cfg)
    perm_rng = np.random.default_rng(cfg["dataset"]["train_seed"])
    perm_indices = perm_rng.permutation(len(train_df))
    shuffled_train = train_df.iloc[perm_indices].reset_index(drop=True)

    for n_samples in cfg["experiments"]["group_A_sample_sizes"]:
        subset = shuffled_train.iloc[: int(n_samples)].reset_index(drop=True)
        for seed in cfg["experiments"]["seeds"]:
            res = train_model(
                train_df=subset, val_df=val_df, hidden_dims=[10],
                config=tcfg, seed=int(seed),
                checkpoint_path=MODELS / f"A_{int(n_samples):04d}_seed{seed}.pt",
            )
```

Log kết quả thực thi Code 3.1:

```
A_0200_seed42: val_MSE=0.002025355, epoch=100, seconds=0.27
A_0200_seed123: val_MSE=0.001258951, epoch=100, seconds=0.27
A_0200_seed2026: val_MSE=0.002298082, epoch=100, seconds=0.27
A_0400_seed42: val_MSE=0.001705238, epoch=99, seconds=0.37
A_0400_seed123: val_MSE=0.0009073949, epoch=97, seconds=0.37
A_0400_seed2026: val_MSE=0.001449791, epoch=100, seconds=0.37
```

A_0600_seed42: val_MSE=0.001105305, epoch=98, seconds=0.46

A_0600_seed123: val_MSE=0.0008370293, epoch=94, seconds=0.45

A_0600_seed2026: val_MSE=0.001002994, epoch=100, seconds=0.53

A_0800_seed42: val_MSE=0.001223693, epoch=98, seconds=0.54

A_0800_seed123: val_MSE=0.0007898718, epoch=91, seconds=0.55

A_0800_seed2026: val_MSE=0.000783782, epoch=97, seconds=0.55

A_1000_seed42: val_MSE=0.0008749529, epoch=100, seconds=0.62

A_1000_seed123: val_MSE=0.0007547017, epoch=100, seconds=0.62

A_1000_seed2026: val_MSE=0.0007599073, epoch=100, seconds=0.60

A_1200_seed42: val_MSE=0.0007470031, epoch=100, seconds=0.70

A_1200_seed123: val_MSE=0.0006508162, epoch=100, seconds=0.69

A_1200_seed2026: val_MSE=0.0006590858, epoch=98, seconds=0.69

A_1400_seed42: val_MSE=0.0006571295, epoch=100, seconds=0.79

A_1400_seed123: val_MSE=0.0005706916, epoch=100, seconds=0.79

A_1400_seed2026: val_MSE=0.0005851139, epoch=98, seconds=0.82

A_1600_seed42: val_MSE=0.0005708103, epoch=100, seconds=0.97

A_1600_seed123: val_MSE=0.0003771637, epoch=100, seconds=0.92

A_1600_seed2026: val_MSE=0.0005293263, epoch=99, seconds=0.88

A_1800_seed42: val_MSE=0.0006136706, epoch=95, seconds=0.96

A_1800_seed123: val_MSE=0.0008067995, epoch=54, seconds=0.70

A_1800_seed2026: val_MSE=0.0005075283, epoch=95, seconds=0.97

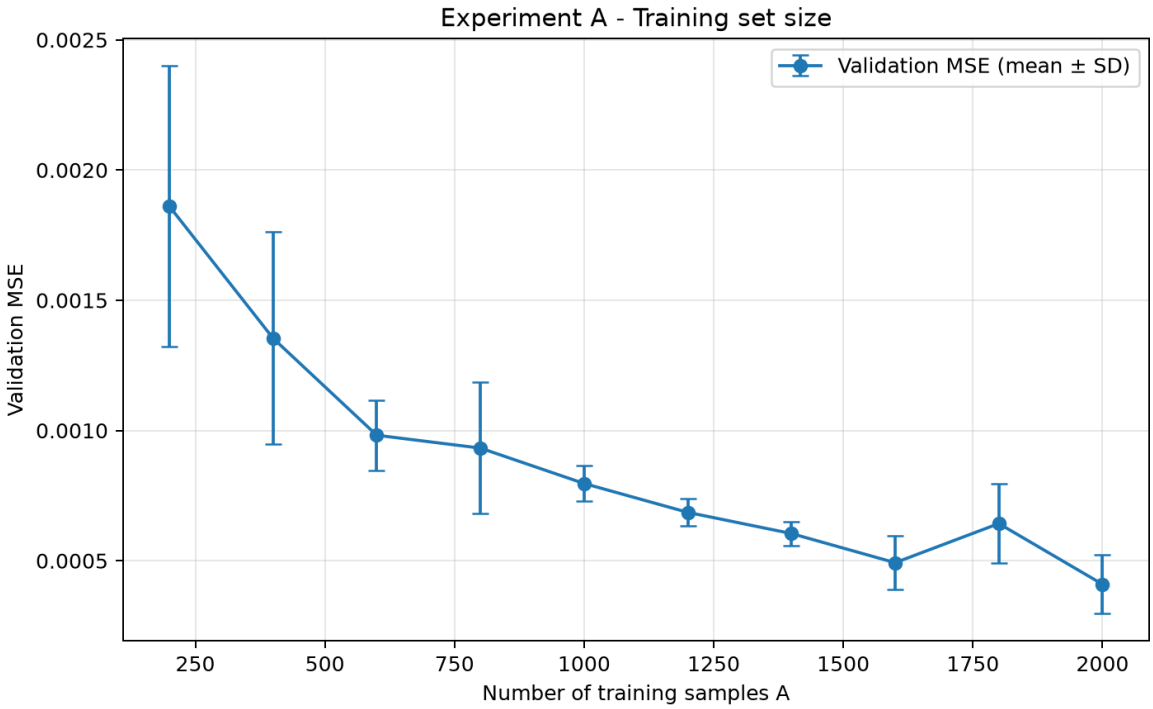
A_2000_seed42: val_MSE=0.0004850698, epoch=99, seconds=1.04

A_2000_seed123: val_MSE=0.0002796093, epoch=98, seconds=1.03

A_2000_seed2026: val_MSE=0.000464144, epoch=100, seconds=1.04

Số mẫu A	Train MSE (Mean ± SD)	Validation MSE (Mean ± SD)	Val MAE	Val RMSE	Thời gian
200	0.001755 ± 0.000526	0.001861 ± 0.000539	0.03125	0.04281	0.41s
400	0.001410 ± 0.000434	0.001354 ± 0.000407	0.02761	0.03650	0.56s
600	0.001089 ± 0.000184	0.000982 ± 0.000135	0.02342	0.03128	0.71s

800	0.001000 ± 0.000261	0.000932 ± 0.000252	0.02242	0.03036	0.87s
1000	0.000885 ± 0.000088	0.000797 ± 0.000068	0.02039	0.02821	0.98s
1200	0.000722 ± 0.000067	0.000686 ± 0.000053	0.01943	0.02617	1.12s
1400	0.000636 ± 0.000044	0.000604 ± 0.000046	0.01794	0.02457	1.30s
1600	0.000526 ± 0.000110	0.000492 ± 0.000102	0.01656	0.02211	1.46s
1800	0.000711 ± 0.000176	0.000643 ± 0.000152	0.01893	0.02524	1.39s
2000	0.000430 ± 0.000106	0.000410 ± 0.000113	0.01551	0.02010	1.58s



Hình 3.1. Ảnh hưởng của số lượng mẫu huấn luyện A (200 → 2.000) đến Validation MSE và MAE/RMSE.

Nhận xét Nhóm A: Khi tăng số mẫu Train từ A = 200 lên A = 2.000, Validation MSE trung bình giảm từ 0.001861 ± 0.000539 xuống 0.000410 ± 0.000113 (giảm 77,97%), đồng thời độ lệch chuẩn giữa các seed giảm từ 0.000539 xuống 0.000113. Điều này khẳng định dữ liệu huấn luyện dày hơn giúp mô hình bao phủ tốt vùng đỉnh và sườn hàm Gaussian, tăng độ chính xác và độ ổn định, đổi lại thời gian huấn luyện tăng tuyến tính từ 0.41s lên 1.58s.

3.2. Experiment B – Ảnh hưởng của số lượng neuron ẩn (1 tầng ẩn)

Cố định tập Train A = 2.000 mẫu và số tầng ẩn C = 1. Thay đổi số neuron ẩn B $\in$ {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} (kiến trúc 2-B-1, số tham số tăng từ 5 đến 41).

Code 3.2. Vòng lặp thực nghiệm Nhóm B

```
def run_group_b(cfg: dict, train_df: pd.DataFrame, val_df: pd.DataFrame) -> None:
    tcfg = make_training_config(cfg)
    for width in cfg["experiments"]["group_B_hidden_neurons"]:
        for seed in cfg["experiments"]["seeds"]:
            res = train_model(
                train_df=train_df, val_df=val_df, hidden_dims=[int(width)],
                config=tcfg, seed=int(seed),
                checkpoint_path=MODELS / f"B_{int(width):04d}_seed{seed}.pt",
            )
```

Log kết quả thực thi Code 3.2 (30 lượt chạy trong logs/B_console.txt):

```
B_0001_seed42: val_MSE=0.003197854, epoch=63, seconds=0.82
B_0001_seed123: val_MSE=0.003199241, epoch=16, seconds=0.36
B_0001_seed2026: val_MSE=0.003195577, epoch=86, seconds=0.99
B_0002_seed42: val_MSE=0.002295951, epoch=100, seconds=1.12
B_0002_seed123: val_MSE=0.002662354, epoch=100, seconds=1.06
B_0002_seed2026: val_MSE=0.002358196, epoch=100, seconds=1.05
B_0003_seed42: val_MSE=0.002140156, epoch=100, seconds=1.06
B_0003_seed123: val_MSE=0.001558132, epoch=100, seconds=1.09
B_0003_seed2026: val_MSE=0.002712197, epoch=100, seconds=1.03
B_0004_seed42: val_MSE=0.0009006817, epoch=100, seconds=1.07
B_0004_seed123: val_MSE=0.001651375, epoch=100, seconds=1.04
B_0004_seed2026: val_MSE=0.001661116, epoch=75, seconds=1.03
B_0005_seed42: val_MSE=0.0006495673, epoch=98, seconds=1.05
B_0005_seed123: val_MSE=0.0007445635, epoch=100, seconds=1.03
B_0005_seed2026: val_MSE=0.0006972042, epoch=99, seconds=1.05
```

B_0006_seed42: val_MSE=0.000711568, epoch=98, seconds=1.07

B_0006_seed123: val_MSE=0.0007371816, epoch=97, seconds=1.08

B_0006_seed2026: val_MSE=0.0006927359, epoch=95, seconds=1.06

B_0007_seed42: val_MSE=0.0007148689, epoch=94, seconds=1.05

B_0007_seed123: val_MSE=0.0007190707, epoch=97, seconds=1.03

B_0007_seed2026: val_MSE=0.0004211567, epoch=100, seconds=1.03

B_0008_seed42: val_MSE=0.0005709247, epoch=97, seconds=1.05

B_0008_seed123: val_MSE=0.0007287645, epoch=100, seconds=1.06

B_0008_seed2026: val_MSE=0.0008141966, epoch=100, seconds=1.09

B_0009_seed42: val_MSE=0.0004043161, epoch=99, seconds=1.09

B_0009_seed123: val_MSE=0.0007251327, epoch=98, seconds=1.15

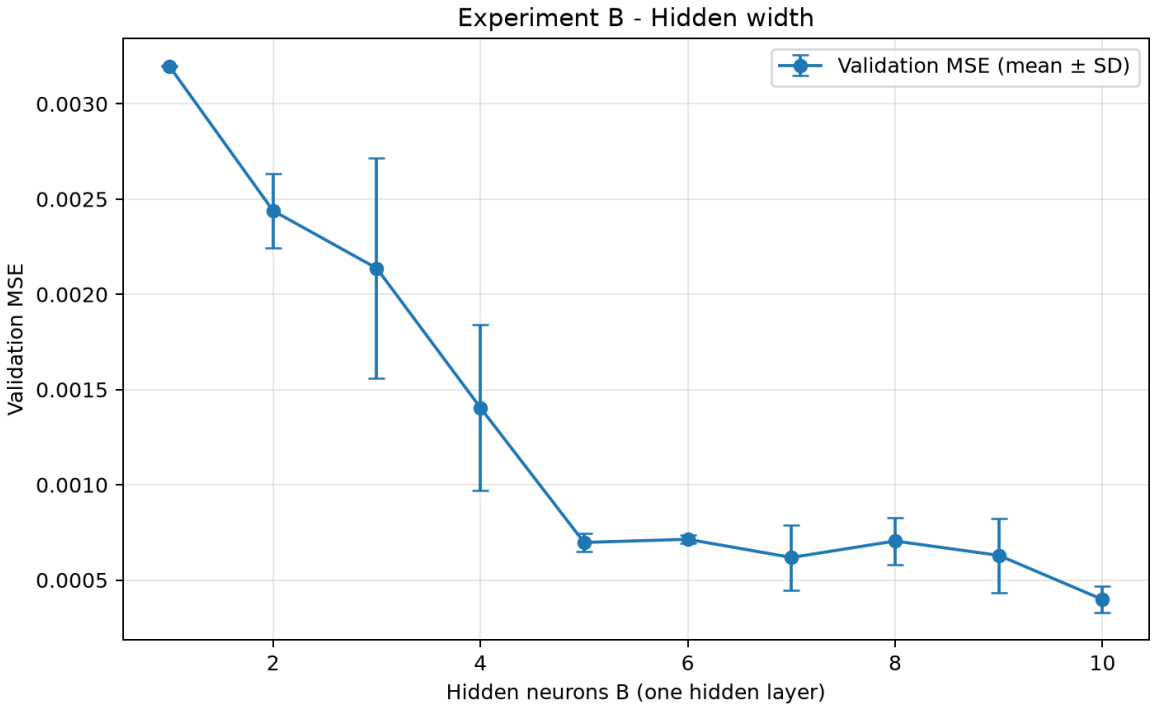
B_0009_seed2026: val_MSE=0.0007584217, epoch=98, seconds=1.10

B_0010_seed42: val_MSE=0.0004603308, epoch=99, seconds=1.08

B_0010_seed123: val_MSE=0.0003217573, epoch=99, seconds=1.07

B_0010_seed2026: val_MSE=0.0004110766, epoch=99, seconds=1.06

Độ rộng B	Train MSE (Mean ± SD)	Validation MSE (Mean ± SD)	Val MAE	Val RMSE
B = 1 (5p)	0.003380 ± 0.000001	0.003198 ± 0.000002	0.03989	0.05655
B = 2 (9p)	0.002599 ± 0.000180	0.002439 ± 0.000196	0.03415	0.04936
B = 3 (13p)	0.002232 ± 0.000618	0.002137 ± 0.000577	0.03186	0.04594
B = 4 (17p)	0.001489 ± 0.000440	0.001404 ± 0.000436	0.02606	0.03714
B = 5 (21p)	0.000752 ± 0.000059	0.000697 ± 0.000047	0.01997	0.02639
B = 6 (25p)	0.000777 ± 0.000033	0.000714 ± 0.000022	0.02023	0.02672
B = 7 (29p)	0.000668 ± 0.000179	0.000618 ± 0.000171	0.01878	0.02469
B = 8 (33p)	0.000770 ± 0.000130	0.000705 ± 0.000123	0.01977	0.02647
B = 9 (37p)	0.000673 ± 0.000207	0.000629 ± 0.000196	0.01892	0.02486
B = 10 (41p)	0.000420 ± 0.000063	0.000398 ± 0.000070	0.01524	0.01989



Hình 3.2. Ảnh hưởng của số lượng neuron ẩn B (1 → 10) trong mạng 1 tầng ẩn đến Validation MSE và MAE/RMSE.

Nhận xét Nhóm B: Ở B = 1..3 neuron, cả Train MSE (0.003380 → 0.002232) và Validation MSE (0.003198 → 0.002137) đều rất cao do hiện tượng underfitting, vì 1 đến 3 hàm Tanh một chiều không thể khép kín mặt chuông Gaussian 2 chiều. Khi tăng độ rộng lên B ≥ 5, Validation MSE giảm mạnh xuống 0.000697 (B = 5) và đạt thấp nhất ở B = 10 với Validation MSE = 0.000398 ± 0.000070 (giảm 87,55% so với B = 1). Trong toàn miền B ≤ 10, Train MSE và Validation MSE bám sát nhau, không có dấu hiệu overfitting.

3.3. Experiment C – Ảnh hưởng của số lượng tầng ẩn (Độ sâu mạng)

Cố định A = 2.000 mẫu Train và số neuron mỗi tầng ẩn bằng 10. Thay đổi số lượng tầng ẩn C ∈ {1, 2, 3, 4}, tương ứng với các kiến trúc 2-10-1 (41 tham số), 2-10-10-1 (151 tham số), 2-10-10-10-1 (261 tham số) và 2-10-10-10-10-1 (371 tham số).

Code 3.3. Vòng lặp thực nghiệm Nhóm C

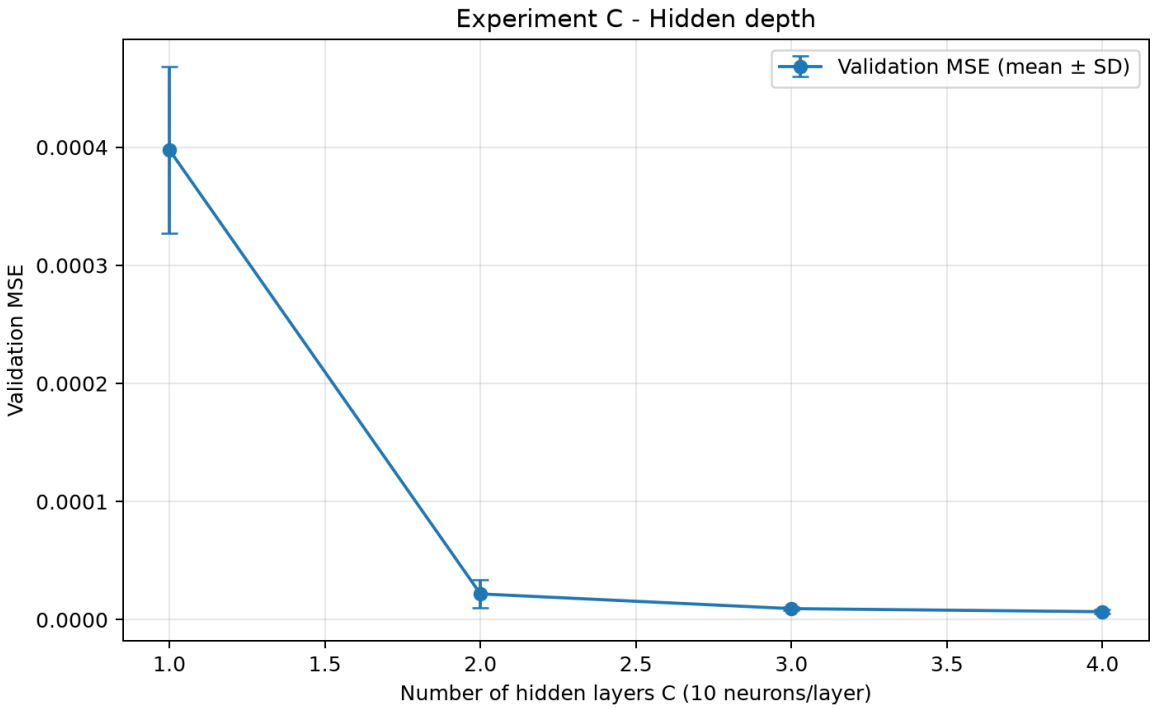
```
def run_group_c(cfg: dict, train_df: pd.DataFrame, val_df: pd.DataFrame) -> None:
    tcfg = make_training_config(cfg)
    width = int(cfg["experiments"]["group_C_fixed_neurons"])
    for depth in cfg["experiments"]["group_C_hidden_layers"]:
        hidden_dims = [width] * int(depth)
        for seed in cfg["experiments"]["seeds"]:
            res = train_model(
                train_df=train_df, val_df=val_df, hidden_dims=hidden_dims,
                config=tcfg, seed=int(seed),
                checkpoint_path=MODELS / f"C_{int(depth):04d}_seed{seed}.pt",
            )
```

Log kết quả thực thi Code 3.3:

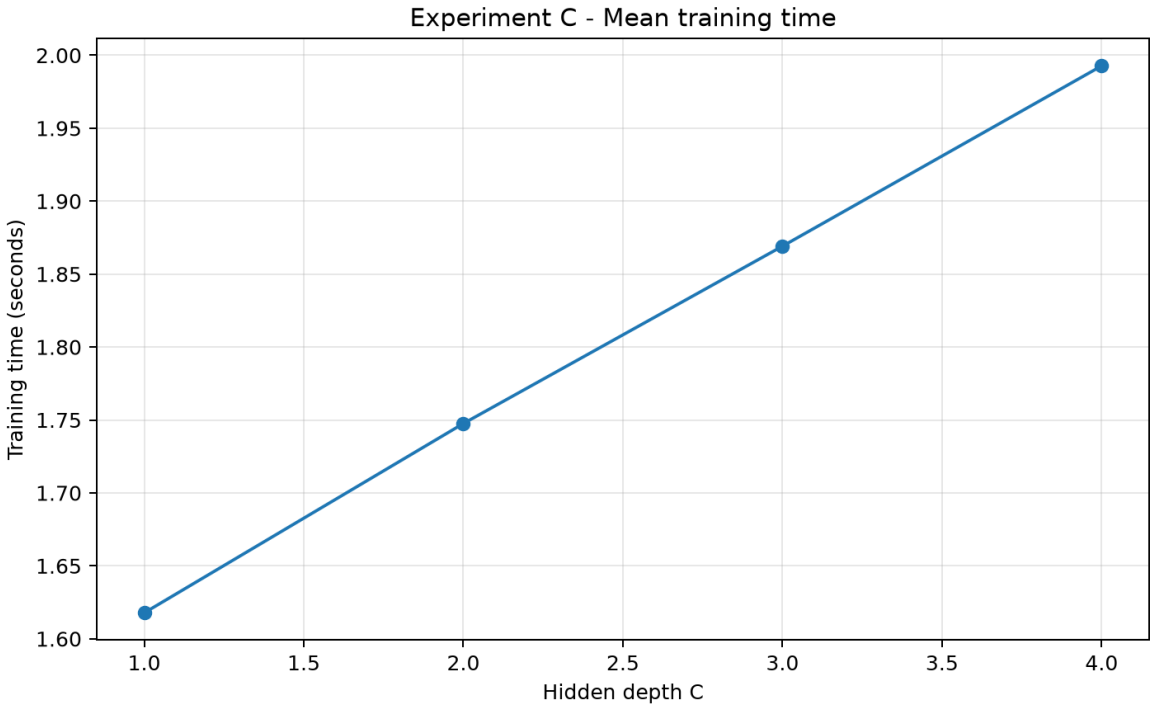
```
C_0001_seed42: val_MSE=0.0004603308, epoch=99, seconds=1.14
C_0001_seed123: val_MSE=0.0003217573, epoch=99, seconds=1.09
C_0001_seed2026: val_MSE=0.0004110766, epoch=99, seconds=1.06
C_0002_seed42: val_MSE=2.099202e-05, epoch=89, seconds=1.24
C_0002_seed123: val_MSE=3.427285e-05, epoch=100, seconds=1.22
C_0002_seed2026: val_MSE=1.032462e-05, epoch=91, seconds=1.18
C_0003_seed42: val_MSE=8.414831e-06, epoch=94, seconds=1.29
C_0003_seed123: val_MSE=1.076932e-05, epoch=97, seconds=1.23
C_0003_seed2026: val_MSE=9.013329e-06, epoch=96, seconds=1.22
C_0004_seed42: val_MSE=6.738205e-06, epoch=85, seconds=1.32
C_0004_seed123: val_MSE=8.647927e-06, epoch=99, seconds=1.33
```

C_0004_seed2026: val_MSE=5.56347e-06, epoch=71, seconds=1.22

Độ sâu C	Train MSE (Mean ± SD)	Validation MSE (Mean ± SD)	Val MAE	Thời gian
C = 1 (41p)	0.0004202 ± 0.0000631	0.0003977 ± 0.0000702	0.015244	1.62s
C = 2 (151p)	0.0000207 ± 0.0000104	0.0000219 ± 0.0000120	0.003575	1.75s
C = 3 (261p)	0.0000089 ± 0.0000013	0.0000094 ± 0.0000012	0.002420	1.87s
C = 4 (371p)	0.0000066 ± 0.0000016	0.0000068 ± 0.0000018	0.002065	1.99s



Hình 3.3. Ảnh hưởng của số tầng ẩn C (1 → 4) đến Validation MSE và MAE/RMSE.



Hình 3.4. Thời gian huấn luyện trung bình theo số tầng ẩn C = 1..4.

Nhận xét Nhóm C: Việc tăng độ sâu từ C = 1 lên C = 2 tầng ẩn giúp Validation MSE giảm 18,2 lần (từ 3.9815e-04 xuống 2.1869e-05). Tại C = 4 tầng ẩn (2-10-10-10-1), Validation MSE đạt mức thấp nhất toàn bộ nghiên cứu là 6.7960e-06 ± 1.8237e-06 (giảm 58,6 lần so với mạng nông C = 1), trong khi thời gian huấn luyện trung bình chỉ tăng nhẹ từ 1.62s lên 1.99s.

3.4. Lựa chọn mô hình cuối, nhật ký thực nghiệm và đánh giá Test

Code 3.4. Lựa chọn kiến trúc tốt nhất theo Validation MSE và huấn luyện lại mô hình cuối

```
candidate_df = pd.concat([sum_b, sum_c], ignore_index=True)
candidate_df = candidate_df.sort_values(["val_mse_mean",
"param_count"]).reset_index(drop=True)
best_row = candidate_df.iloc[0]  # Selected: group='C', factor_value=4,
hidden_dims=[10, 10, 10, 10]

final_res = train_model(
    train_df=train_df, val_df=val_df, hidden_dims=[10, 10, 10, 10],
```

```
    config=final_cfg, seed=999, checkpoint_path=MODELS / "best_model.pt",
)
```

Log kết quả thực thi Code 3.4:

```
Sổ nhật ký Excel: reports/experiment_journal.xlsx (9 sheets)
Sheets: ['Tong_quan', 'A_So_mau', 'B_Do_rong', 'C_Do_sau', 'Run_detail',
'Epoch_history', 'Final_model', 'Final_epoch', 'Nhat_ky']

Trích 7 mốc epoch từ logs/final_model_epoch_history.csv:
 epoch  train_mse   val_mse   val_mae  val_rmse
     1   0.075054  0.005278  0.064556  0.072646
    10   0.000644  0.000562  0.019572  0.023706
    50   0.000041  0.000037  0.004987  0.006119
   100   0.000026  0.000022  0.003843  0.004727
   150   0.000016  0.000017  0.003304  0.004063
   208   0.000021  0.000008  0.002151  0.002771  (Best Validation Checkpoint)
   248   0.000025  0.000056  0.006678  0.007463  (Early Stopping)
```

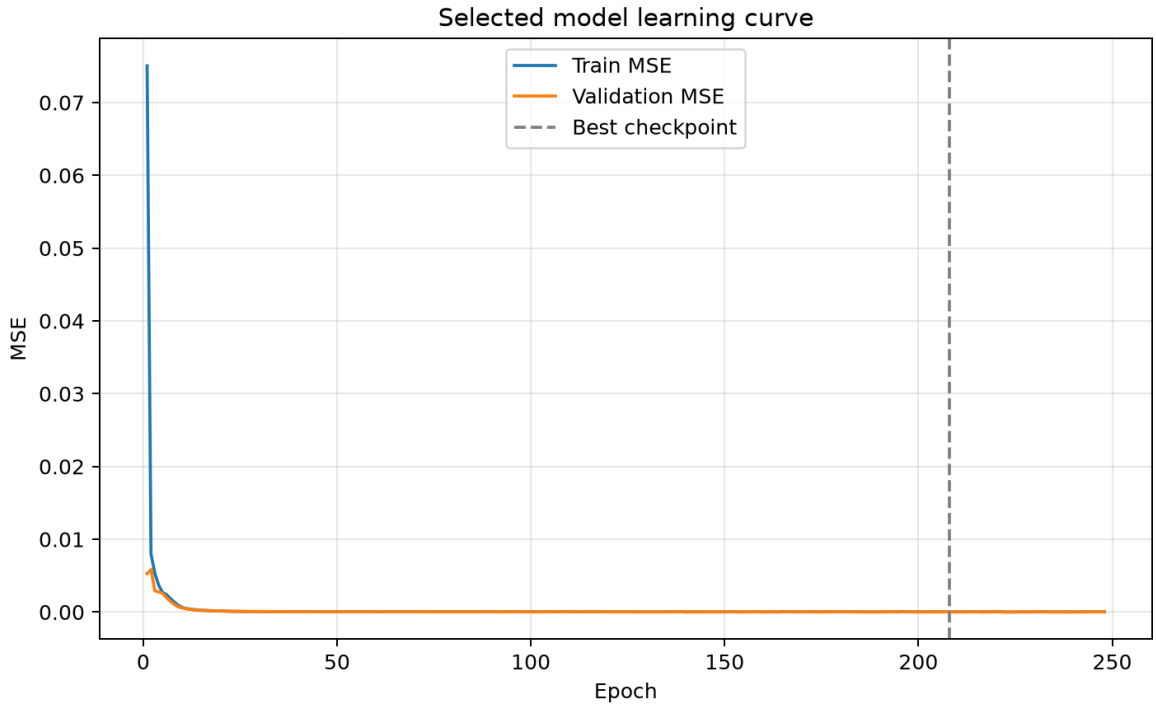
Code 3.5. Vẽ đường cong hội tụ và đối chiếu y_true vs y_pred trên tập Test

```
fig, ax = plt.subplots(figsize=(7.6, 4.2))
ax.plot(final_hist["epoch"], final_hist["train_mse"], label="Train MSE", lw=1.8)
ax.plot(final_hist["epoch"], final_hist["val_mse"], label="Validation MSE", lw=1.8)
ax.axvline(best_ep, ls="--", lw=1.3, label=f"Best validation epoch = {best_ep}")
ax.set_yscale("log"); ax.set_xlabel("Epoch"); ax.set_ylabel("MSE (log scale)")
fig.savefig("figures/11_final_learning_curve.png", dpi=220)
```

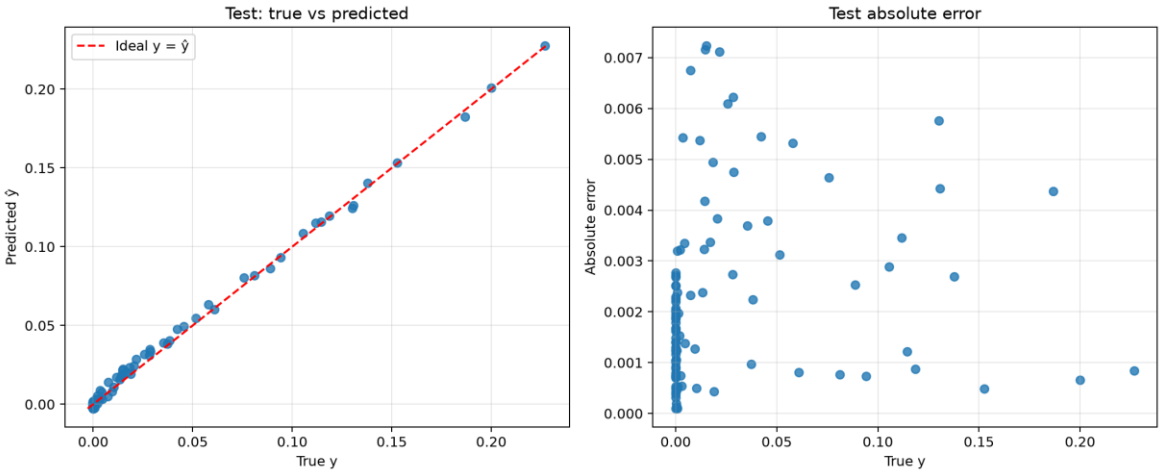
Log kết quả thực thi Code 3.5:

```
10 mẫu đầu tiên trên tập Test (results/final_test_predictions.csv, N=100):
      x1        x2       y_true    y_pred  absolute_error
-0.452991 -0.185976 1.307554e-01  0.126332        0.004423
 0.370227  0.591353 1.144951e-01  0.115714        0.001219
 0.949321  3.805023 2.375081e-07  0.001403        0.001403
 1.939705  0.797826 4.814732e-07 -0.000491        0.000492
 1.368982  3.562594 5.201270e-09  0.000760        0.000760
-0.401678 -0.804100 8.106775e-02  0.081833        0.000765
 1.317194 -3.497814 7.214260e-03  0.004887        0.002328
-0.086246  2.763891 1.401566e-02  0.017252        0.003236
-0.438545 -3.958547 1.575723e-05 -0.002671        0.002686
-1.706249 -3.657564 2.854448e-11  0.000529        0.000529

Kết quả tổng hợp trên 100 mẫu Test độc lập:
Test MSE  = 8.452444e-06 | Test MAE = 0.002277 | Test RMSE = 0.002907
```



Hình 3.5. Đường cong hội tụ Train MSE và Validation MSE của mô hình cuối 2-10-10-10-10-1.



Hình 3.6. Kết quả dự đoán y_true vs y_pred và phân bố sai số tuyệt đối trên 100 mẫu Test.

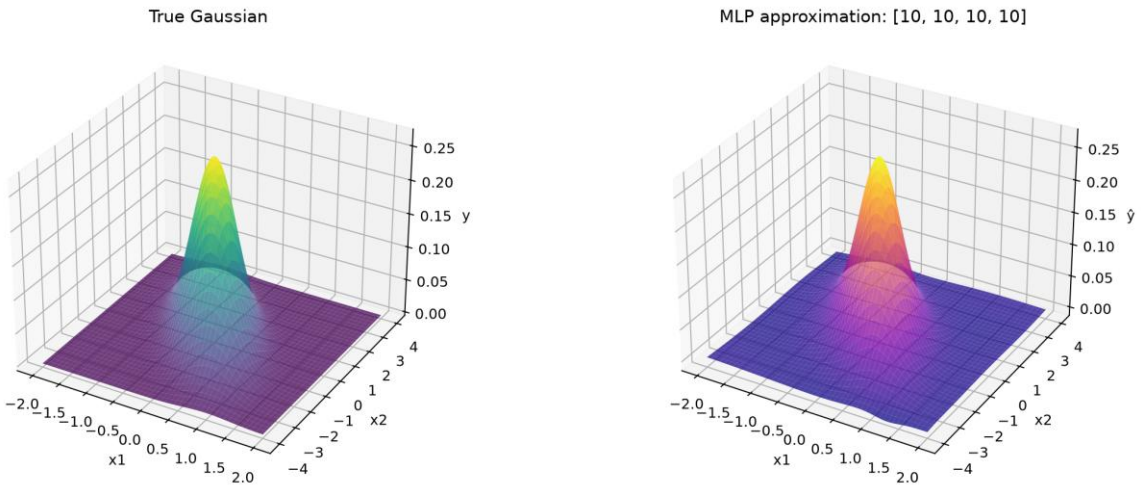
Code 3.6. Tái tạo bề mặt xấp xỉ 3D trên lưới 90×90 và bản đồ nhiệt sai số tuyệt đối

```
grid_pts = np.column_stack([gx1.ravel(), gx2.ravel()])
gz_pred = predict(model, scaler, grid_pts, device="cpu").reshape(gx1.shape)
abs_err = np.abs(gz_true - gz_pred)
fig.savefig("figures/09_true_vs_approx_surface.png", dpi=220)
```

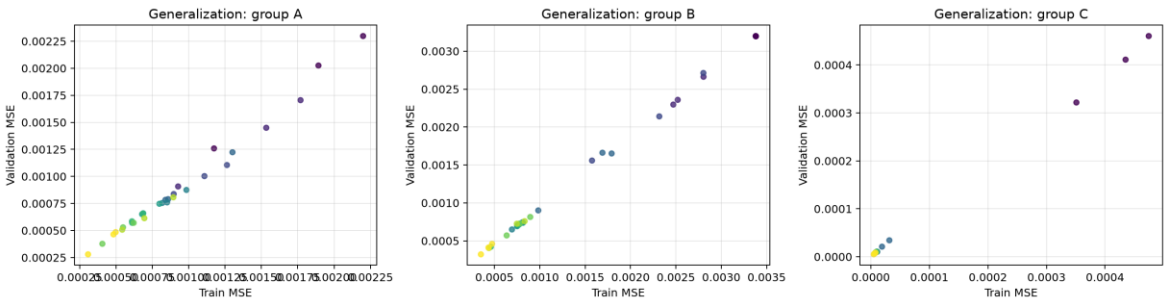
```
fig2.savefig("figures/12_train_validation_tradeoff.png", dpi=220)
```

Log kết quả thực thi Code 3.6:

```
Grid 90x90 (8100 pts) | Mean |y-y_hat| = 0.002210 | Max |y-y_hat| = 0.011052 | Grid MSE = 0.00000836
```



Hình 3.7. Đối chiếu bề mặt Gaussian thật, bề mặt do mạng MLP xấp xỉ và bản đồ nhiệt sai số tuyệt đối.



Hình 3.8. Quan hệ giữa Train MSE và Validation MSE trên 72 lượt chạy thực nghiệm A, B, C.

IV - DISCUSSION AND CONCLUSION

4.1. Thảo luận kết quả thực nghiệm

Khía cạnh phân tích	Kết quả định lượng	Giải thích bản chất
Ảnh hưởng của số mẫu Train (Nhóm A)	Validation MSE giảm 77,97% (từ 1.861e-03 ở A=200 xuống 4.100e-04 ở A=2.000).	Khi số mẫu tăng lên 2.000, mật độ điểm phủ kín vùng đỉnh và sườn Gaussian giúp giảm phương sai ước lượng.
Số neuron ẩn & Underfitting (Nhóm B)	B=1..3 bị underfitting (Val MSE 3.198e-03 → 2.137e-03); đạt 3.98e-04 tại B=10.	Cần ít nhất 5–10 hàm Tanh định hướng khác nhau giao thoa lại để khép kín đường đồng mức elip nghiêng.
Ảnh hưởng của độ sâu mạng (Nhóm C)	Tăng độ sâu từ C=1 lên C=4 giảm Validation MSE 58,6 lần (xuống 6.796e-06).	Mạng nhiều tầng ẩn phân tách tự nhiên phép biến đổi toàn phương bậc hai và phép uốn hàm mũ.
Chênh lệch Train – Validation	Train MSE = 8.01e-06, Val MSE = 7.68e-06, Test MSE = 8.45e-06.	Nhờ chuẩn hóa Z-score, weight decay 1e-5 và early stopping, mô hình không bị overfitting.
Độ ổn định giữa các hạt giống	Cấu hình C=4 đạt Val MSE = 6.80e-06 ± 1.82e-06 qua 3 seed (42, 123, 2026).	Khởi tạo Xavier kết hợp chuẩn hóa đầu vào giúp mạng hội tụ ổn định qua các lần chạy độc lập.

4.2. Kết luận và hướng phát triển

- Bài tập lớn đã hoàn thành toàn bộ yêu cầu bằng PyTorch thuần túy. Kiến trúc được lựa chọn dựa trên Validation MSE là mạng MLP 4 tầng ẩn × 10 neuron (2-10-10-10-

10-1, 371 tham số), đạt Test MSE = 8.4524×10^{-6}, Test MAE = 0.002277 và Test RMSE = 0.002907 trên 100 mẫu Test độc lập.

- Hạn chế: Tập Test cố định 100 mẫu theo đề bài; ngân sách tìm kiếm A/B/C giới hạn ở 100 epoch; các yếu tố A, B, C được khảo sát đơn biến.
- Hướng phát triển: Bổ sung Learning Rate Scheduler (CosineAnnealingLR hoặc ReduceLROnPlateau), khảo sát các hàm kích hoạt trơn như GELU/SiLU và áp dụng lấy mẫu tập trung quanh vùng gradient lớn.